

Técnico en ingeniería en computación

Diseño de Software Multiplataforma DPS941

Docente: Ing. Alexander Siguenza

Carlos Alberto Cornejo Calderón CC251978

David Antonio Leiva Martínez LM160828

Moisés David García Casco GC251462

Alcyr Alexander Figueroa Landaverde FL253210

Nelson Mauricio Solano Guardado SG240238

Grupo: DamagePerSecond – T008

Proyecto Foro II

VIDEO

REPOSITORIO

Contenido

1. Introducción 3
2. Firebase y React Native 4
3. Arquitectura de la aplicación móvil 4
4. Configuración de Firebase 5
5. Firebase Authentication 5
5.1 Correo electrónico y contraseña 5
5.2 Inicio de sesión con Google 6
5.3 Sesión y perfil del usuario 6
6. Cloud Firestore 7
7. Cloud Storage 8
8. Firebase Security Rules 9
9. Módulos de la aplicación móvil desarrollados en esta fase 10
10. Conclusiones 11
Referencias 12

1. Introducción

El desarrollo de una aplicación móvil requiere resolver, además de la interfaz, aspectos como la identificación de usuarios, el almacenamiento de datos y el control de acceso a la información. Firebase es una plataforma de Google que ofrece estos servicios de backend listos para usarse mediante sus SDK (Firebase, s. f.). React Native, por su parte, permite construir aplicaciones para Android y iOS con JavaScript y React a partir de una base de código compartida (Meta Open Source, s. f.).

Este documento presenta la investigación sobre la integración de Firebase con React Native y describe cómo se aplicó en la aplicación móvil del sistema de gestión de Recursos Humanos desarrollado por el equipo DamagePerSecond. En ese sistema los empleados solicitan permisos, vacaciones y constancias laborales y los jefes, analistas y gerentes las revisan y las aprueban o rechazan. Se documentan únicamente los servicios de Firebase que el proyecto utiliza: Authentication, Cloud Firestore y Cloud Storage.

2. Firebase y React Native

Firebase es una plataforma de servicios de backend orientada al desarrollo de aplicaciones. En una arquitectura tradicional, la aplicación móvil se comunica con una API, esta con un servidor y el servidor con la base de datos; el equipo debe construir y mantener cada una de esas piezas. Con Firebase, la aplicación se comunica directamente con los servicios mediante el SDK y la seguridad se controla con reglas definidas en la plataforma (Firebase, s. f.).

React Native, por sí solo, no incluye un backend: permite crear una pantalla de inicio de sesión, pero no proporciona la base de datos de usuarios ni el sistema de autenticación (Meta Open Source, s. f.). Firebase cubre esa necesidad, por lo que ambas tecnologías se complementan.

Las razones por las que el equipo eligió esta combinación son las siguientes:

- Desarrollo más rápido: la autenticación, la base de datos y el almacenamiento de archivos ya están construidos y se usan con el SDK.
- Datos en tiempo real: Cloud Firestore permite que la aplicación reciba los cambios de los datos sin tener que volver a consultarlos.
- Seguridad: Authentication identifica a cada usuario y las Security Rules controlan qué puede leer o modificar.
- Menos infraestructura propia: el equipo no administra un servidor para las funciones comunes de la aplicación.

3. Arquitectura de la aplicación móvil

La aplicación móvil se desarrolló con React Native sobre Expo y se conecta a Firebase mediante el SDK de JavaScript. La comunicación se resume así:

Usuario → Aplicación React Native (Expo) → SDK de Firebase → Authentication / Cloud Firestore / Cloud Storage

El código de la aplicación (carpeta mobile/src) se organiza en capas:

- **screens:** pantallas de la aplicación, agrupadas por módulo (auth, permisos, vacaciones, constancias, bandeja, notificaciones, reportes y perfil).
- **context:** AuthContext, que mantiene la sesión y el perfil del usuario y lo comparte con toda la aplicación.
- **services:** funciones que se comunican con Firebase y contienen la lógica de cada módulo.
- **components y navigation:** componentes reutilizables y navegación por pestañas y pilas.

El proyecto también cuenta con una aplicación web desarrollada en Next.js que utiliza Firebase Authentication y Cloud Firestore. Este documento se centra en la aplicación móvil.

4. Configuración de Firebase

La conexión se configura en el archivo services/firebase.js. Los datos del proyecto Firebase se leen desde variables de entorno (EXPO_PUBLIC_…) definidas en un archivo .env local.

```
const app = primeraVez ? initializeApp(firebaseConfig) : getApp();

export const auth = primeraVez
  ? initializeAuth(app, {
      persistence: getReactNativePersistence(AsyncStorage),
    })
  : getAuth(app);

export const db = primeraVez
  ? initializeFirestore(app, { experimentalForceLongPolling: true })
  : getFirestore(app);

export const storage = getStorage(app);
```

La autenticación se inicializa con getReactNativePersistence y AsyncStorage. Con esta configuración la sesión se conserva en el dispositivo y el usuario no tiene que iniciar sesión cada vez que abre la aplicación (Google, s. f.-b).

5. Firebase Authentication

Firebase Authentication gestiona la identidad de los usuarios. Al iniciar sesión correctamente entrega un identificador único (UID), que la aplicación usa para relacionar al usuario con su documento en Firestore y con sus solicitudes (Google, s. f.-b). El proyecto implementa dos métodos de acceso.

5.1 Correo electrónico y contraseña

La pantalla de inicio de sesión valida primero el formulario: el correo es obligatorio y debe tener un formato válido, y la contraseña es obligatoria y debe tener al menos 6 caracteres. Luego, AuthContext llama a signInWithEmailAndPassword y traduce los errores de Firebase a mensajes comprensibles, por ejemplo cuando no hay conexión (auth/network-request-failed) o cuando se hicieron demasiados intentos (auth/too-many-requests).

```
const login = async (correo, clave) => {
  try {
    const cred = await signInWithEmailAndPassword(auth, correo.trim(), clave);
    const perfil = await cargarPerfil(cred.user.uid);
    if (!perfil) {
      await signOut(auth);
      return { ok: false, mensaje: "Este usuario no tiene acceso a la app" };
    }
    setUsuario(perfil);
    return { ok: true };
  } catch (e) {
    ...
  }
};
```

5.2 Inicio de sesión con Google

El inicio de sesión con Google utiliza la biblioteca @react-native-google-signin/google-signin, configurada con el ID de cliente web del proveedor Google habilitado en Firebase. La biblioteca obtiene un idToken de la cuenta de Google y la aplicación lo convierte en una credencial de Firebase con GoogleAuthProvider.credential, que se envía a signInWithCredential.

```
const respuesta = await GoogleSignin.signIn();
const credencial = GoogleAuthProvider.credential(respuesta.data.idToken);
const cred = await signInWithCredential(auth, credencial);
const perfil = await cargarPerfil(cred.user.uid);
if (!perfil) {
  await signOut(auth);
  await GoogleSignin.signOut();
  return { ok: false, mensaje: "Esta cuenta de Google no está registrada en el sistema" };
}
```

En ambos métodos, el acceso exige que el usuario tenga un perfil en Firestore, como se explica a continuación.

5.3 Sesión y perfil del usuario

AuthContext escucha los cambios de sesión con onAuthStateChanged. Cuando hay un usuario autenticado, carga su documento de la colección usuarios (identificado por el UID), que contiene datos como el nombre, el rol y la fotografía. El rol de Firestore se convierte en uno de los roles de la aplicación: Empleado, Jefe inmediato, Analista de nómina, Gerente o Asistente de planilla. Si el documento no existe o el usuario está inactivo, se cierra la sesión y no se permite el acceso. De esta forma, tener una cuenta de Firebase no es suficiente para entrar al sistema: el usuario debe estar registrado y activo.

```
useEffect(() => {
  const cancelar = onAuthStateChanged(auth, async (user) => {
    const perfil = user ? await cargarPerfil(user.uid).catch(() => null) : null;
    setUsuario(perfil);
    set Cargando(false);
  });
  return cancelar;
}, []);
```

6. Cloud Firestore

Cloud Firestore es una base de datos NoSQL que organiza la información en colecciones y documentos (Google, s. f.-a). La aplicación móvil utiliza las siguientes colecciones:

Tabla 1 Colecciones de la aplicación móvil

Colección	Contenido	Quién crea los documentos
usuarios	Perfil de cada usuario: nombre, rol, jefe asignado, fotografía y notificaciones leídas. El identificador del documento es el UID de Authentication.	Administrador de TI
permisos	Solicitudes de permiso: empleado, empresa, motivo, fecha, estado, etapa de aprobación, motivo de rechazo e historial.	El empleado, para sí mismo
vacaciones	Solicitudes de vacaciones del empleado y su estado de aprobación.	El empleado, para sí mismo
constancias	Solicitudes de constancia laboral: tipo, destinatario, motivo, estado, observación y, una vez autorizada, el documento generado.	El empleado, para sí mismo

Los datos se leen en tiempo real con onSnapshot. Un empleado consulta únicamente los documentos cuyo empleadoUid coincide con su UID, mientras que los roles de aprobación consultan la colección completa. Así, cuando una solicitud cambia de estado, la pantalla se actualiza sin que el usuario deba recargarla.

```
export function escucharPermisos(usuario, alCambiar) {
  const consulta =
    usuario.rol === "EMPLEADO"
      ? query(permisosRef, where("empleadoUid", "==", usuario.id))
      : permisosRef;
  return onSnapshot(consulta, (snap) => {
    almacen = snap.docs.map((d) => ({ id: d.id, historial: [], ...d.data() }));
    alCambiar?.();
  }, () => { almacen = []; });
}
```

Para guardar datos, los servicios primero validan la información y después usan addDoc para crear documentos y updateDoc para modificarlos. Por ejemplo, al crear un permiso se validan los campos, se verifica que la fecha no choque con otro permiso del mismo empleado y se asigna la primera etapa de aprobación: si el empleado tiene jefe, la solicitud empieza en el jefe; si no lo tiene, pasa directamente a la analista de nómina.

```
const nueva = {
  empleadoUid: usuario.id,
  empleadoNombre: `${usuario.nombre} ${usuario.apellido}`.trim(),
  jefeUid: usuario.jefeId || null,
  empresa: datos.empresa,
  tipo: "Permiso Personal",
  motivo: datos.motivo.trim(),
  fecha: datos.fecha,
  estado: ESTADOS.PENDIENTE,
  etapaActual: etapaInicialPara(usuario),
  creadaEn: new Date().toISOString(),
  historial: [],
};
const ref = await addDoc(permisosRef, nueva);
```

7. Cloud Storage

Cloud Storage se utiliza para guardar la fotografía de perfil de cada usuario, ya que Firestore está pensado para datos estructurados y Storage para archivos (Google, s. f.-c). El usuario selecciona una imagen de su galería con expo-image-picker. La aplicación la sube a la ruta fotos_perfil/{uid}.jpg, obtiene la URL de descarga y la guarda en el campo fotoUrl de su documento en la colección usuarios.

```
const storageRef = ref(storage, `fotos_perfil/${idUsuario}.jpg`);
await uploadBytes(storageRef, blob);
const downloadURL = await getDownloadURL(storageRef);
await updateDoc(doc(db, "usuarios", idUsuario), { fotoUrl: downloadURL });
```

8. Firebase Security Rules

Las Security Rules determinan quién puede leer o modificar los datos. El proyecto no deja Firestore en modo de prueba: el archivo firestore.rules define reglas por colección basadas en el rol del usuario autenticado. Una función, rolActual(), consulta el rol en el documento usuarios/{uid} de quien hace la petición, y otra, esRolAdministrativo(), agrupa los roles que participan en las aprobaciones (Administrador de TI, Gerente, Analista de nómina, Asistente de planilla y Jefe inmediato) (Google, s. f.-d).

Las reglas aplicadas a cada colección son:

- **Usuarios:** cada persona lee su propio documento; el Administrador de TI y el Asistente de planilla pueden leer los de todos; solo el Administrador de TI crea usuarios y nadie los elimina. Un usuario solo puede modificar los campos fotoUrl y notificacionesLeidas de su propio documento.
- **Permisos y vacaciones:** el dueño lee las suyas y los roles administrativos leen todas; cada empleado solo puede crear solicitudes a su nombre; la decisión corresponde a los roles administrativos. En permisos, el dueño también puede corregir una solicitud que fue rechazada.
- **Constancias:** el dueño lee las suyas y los roles administrativos leen todas; solo el Asistente de planilla y el Gerente pueden autorizarlas.

```
match /permisos/{permisoId} {
  allow read: if request.auth != null &&
    (resource.data.empleadoUid == request.auth.uid || esRolAdministrativo());
  allow create: if request.auth != null &&
    request.resource.data.empleadoUid == request.auth.uid;
  allow update: if request.auth != null &&
    ((resource.data.empleadoUid == request.auth.uid && resource.data.estado == 'Rechazada') ||
     esRolAdministrativo());
  allow delete: if false;
}
```

La seguridad no depende solo de la interfaz: ocultar un botón en la aplicación no impide una operación. Por eso los permisos se verifican también en Firestore.

9. Módulos de la aplicación móvil desarrollados en esta fase

Los siguientes módulos de la aplicación móvil se conectan a Firebase:

Tabla 2 Módulos de la aplicación móvil

Módulo	Funcionalidad	Servicios de Firebase
Inicio de sesión	Acceso con correo y contraseña o con cuenta de Google, con validación del formulario y control de acceso por perfil activo.	Authentication, Firestore
Permisos	Solicitud de permisos con validaciones, historial y estado de cada solicitud; la solicitud avanza por etapas de aprobación y, si se rechaza, el empleado puede corregirla.	Firestore
Vacaciones	Solicitud de vacaciones con cálculo del saldo disponible (15 días hábiles anuales), conteo de días hábiles y validación de que las fechas no inicien ni terminen en domingo o feriado.	Firestore
Constancias	Solicitud de constancia laboral, autorización por Recursos Humanos o la Gerencia, cálculo de la antigüedad y generación de un PDF con código QR de verificación, que se puede imprimir o compartir.	Firestore
Bandeja de aprobaciones	Lista de solicitudes pendientes para el jefe inmediato, la analista, el gerente y Recursos Humanos, con detalle y decisión (aprobar o rechazar).	Firestore
Notificaciones	Avisos dentro de la aplicación sobre el estado de las solicitudes; las notificaciones ya vistas se guardan en el perfil del usuario.	Firestore
Reportes	Reporte de días de vacaciones registrados por departamento.	Firestore
Perfil	Consulta de los datos del usuario y cambio de la fotografía de perfil.	Storage, Firestore

10. Conclusiones

La combinación de React Native y Firebase permitió construir la aplicación móvil sin desarrollar ni mantener un servidor propio. Authentication resolvió el acceso con correo y con Google, Cloud Firestore almacenó las solicitudes y las mantuvo actualizadas en tiempo real, y Cloud Storage guardó las fotografías de perfil. Las Security Rules por rol garantizan que cada usuario acceda solo a la información que le corresponde.

Como parte del avance del proyecto, la aplicación móvil ya cubre el inicio de sesión, las solicitudes de permisos, vacaciones y constancias, la bandeja de aprobaciones, las notificaciones, los reportes y el perfil.

Referencias

Firebase. (s. f.). *Firebase documentation*. https://firebase.google.com/docs

Google. (s. f.-a). *Cloud Firestore documentation*. Firebase. https://firebase.google.com/docs/firestore

Google. (s. f.-b). *Firebase Authentication*. Firebase. https://firebase.google.com/docs/auth

Google. (s. f.-c). *Firebase Cloud Storage documentation*. Firebase. https://firebase.google.com/docs/storage

Google. (s. f.-d). *Firebase Security Rules*. Firebase. https://firebase.google.com/docs/rules

Meta Open Source. (s. f.). *React Native documentation*. https://reactnative.dev/docs/getting-started